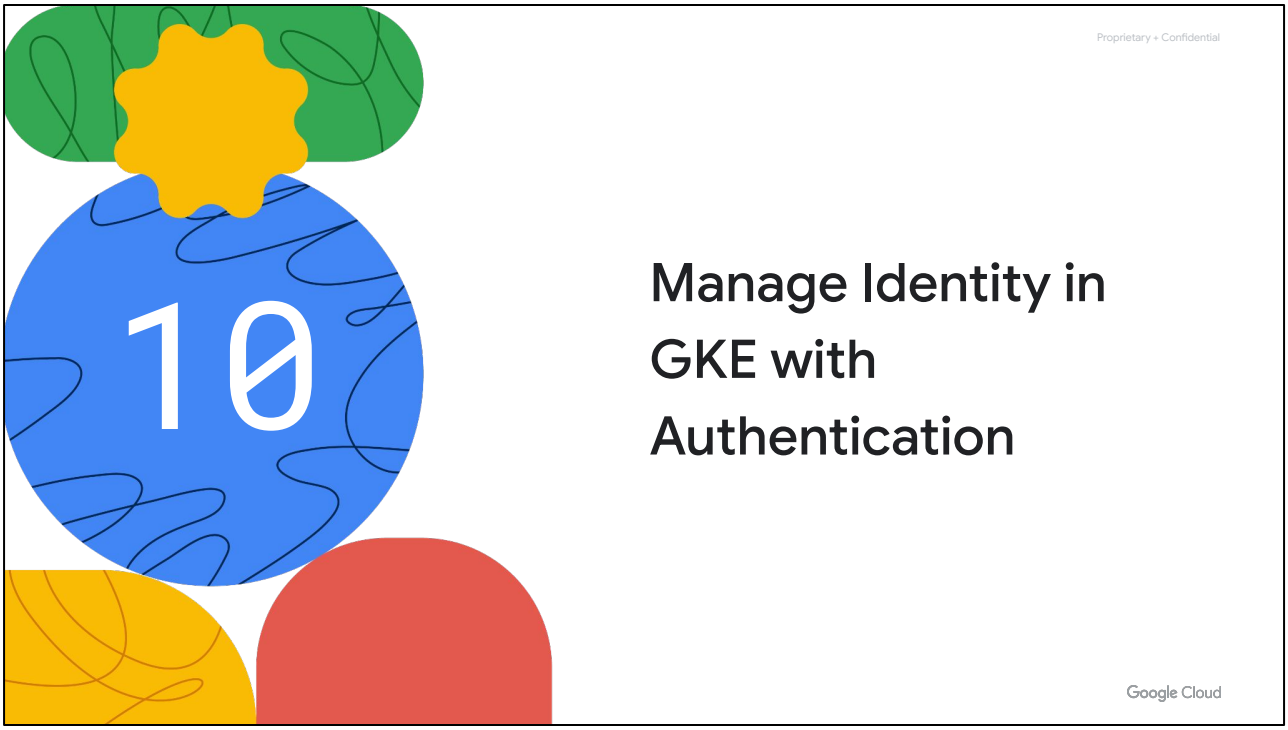


Identity and access control management is essential for any Kubernetes environment.

GKE offers a robust and flexible system to handle these complexities across various deployment scenarios.

Let’s examine the different approaches to identity management in GKE, from basic authentication to advanced federation.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

06 Fleet Workload Identity

Google Cloud

In this section, titled Manage identity in GKE with authentication, you learn to:

- Summarize the differences between authentication methods for GKE clusters and explain when to use each.
- Summarize the key features of connect gateway and explain how it simplifies and secures connections to GKE fleet member clusters.
- Configuring connect gateway for authentication and authorization.
- Securely access clusters and provide authentication using OpenID Connect (OIDC) and third-party identity providers (IdPs).
- Given a GKE cluster and a third-party identity provider (IdP), configure GKE Identity Service to enable authentication and authorization for users.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

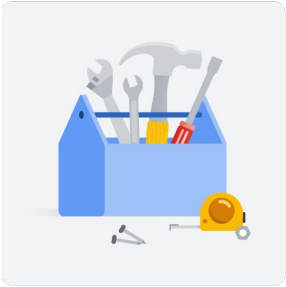
06 Fleet Workload Identity

Google Cloud

Open-source Kubernetes supports basic username/password authentication and Kubernetes Service Account (KSA) tokens, but these may be insufficient for complex security needs.

Proprietary + Confidential

Kubernetes service account (KSA) authentication



Kubernetes service accounts

- A KSA provides an identity for processes running in a Pod and is assigned when a Pod is created.
- The KSA provides the Pod with credentials for authentication when accessing the Kubernetes API server and other services.

Google Cloud

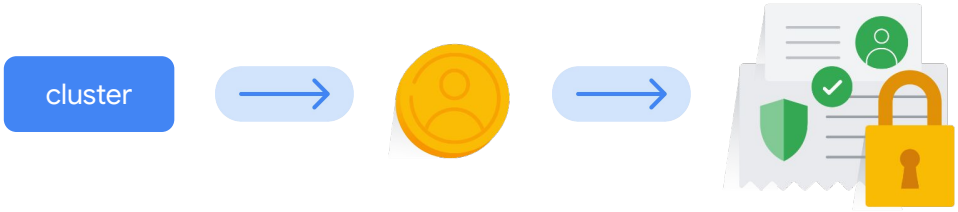
Instead, you can use KSA authentication, which is a fundamental tool for identity management within a cluster.

A KSA provides an identity for processes running in a Pod and is assigned when a Pod is created.

The KSA provides the Pod with credentials for authentication when accessing the Kubernetes API server and other services.

Proprietary + Confidential

Use bearer tokens when Google-provided authentication solutions are not suitable

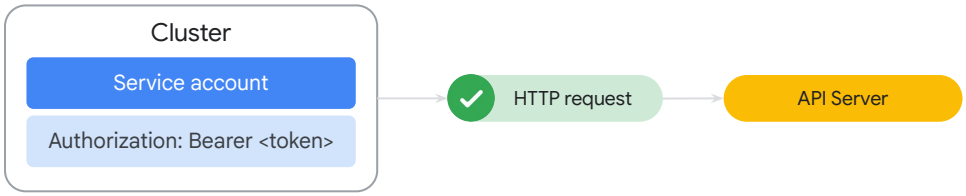


Google Cloud

There may be situations where Google-provided authentication solutions, like gcloud, are not feasible or suitable for your specific needs. In this case, you can use a Kubernetes cluster to authenticate with bearer tokens.

Proprietary + Confidential

How do bearer tokens work?



Google Cloud

When using bearer tokens, a service account is created in your Kubernetes cluster and assigned resource access permissions.

The service account's secret contains a token that can be retrieved and used in HTTP requests to the Kubernetes API server.

The format is typically: Authorization: Bearer <token>.

Proprietary + Confidential

Bearer token benefits

Simple

They are easy to set up and use.

Flexible

They are useful for custom applications or scripts interacting with the cluster.

Automation

They can be integrated into automated workflows and CI/CD pipelines.

Google Cloud

Bearer tokens are simple, flexible, and can be automated.

They are relatively easy to set up and use, useful for custom applications or scripts interacting with the cluster, and can be integrated into automated workflows and CI/CD pipelines.

Proprietary + Confidential

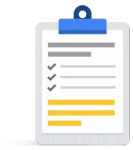
Bearer token considerations

Security

If compromised, bearer tokens grant access to the cluster using the permissions of the service account.

Expiration

Kubernetes tokens have an expiration time, so you must manage token renewal.



Tracking actions

Tracking actions performed with the service account can be challenging.

Google Cloud

There are few key considerations when using bearer tokens.

First, security is paramount. Securely store and manage the bearer token. If compromised, it grants access to the cluster using the permissions of the service account.

Kubernetes tokens have an expiration time, so you must manage token renewal.

And tracking actions performed with the service account can be more challenging compared to user-based authentication.

Proprietary + Confidential

Identity management can be challenging

As organizations grow, bearer tokens may no longer be suitable for authentication.

Centralized control and consistent policies are essential for security and operational efficiency.

Google Cloud

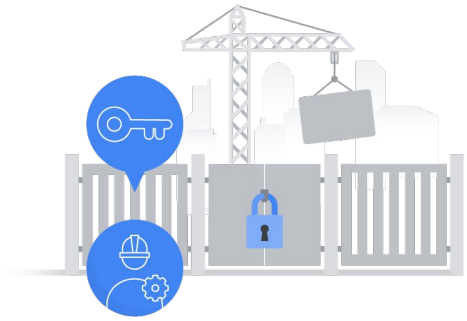
Managing identity and setting up permissions for Kubernetes clusters can be a challenge, especially as the amount of clusters and users grow. As this growth occurs, bearer tokens may no longer be suitable for authentication.

Different tokens are required to authenticate to each cluster. These tokens must be managed, which adds management overhead.

Centralized control and consistent policies are essential for security and operational efficiency.

Proprietary + Confidential

Authentication with Google Identity and IAM



Google Identity enables you to leverage IAM permissions for granular control over who can access and manage your GKE clusters within Google Cloud.

Google Cloud

To solve this challenge, you can authenticate with your Google Identity in GKE. Google Identity enables you to leverage IAM permissions for granular control over who can access and manage your GKE clusters within Google Cloud.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

06 Fleet Workload Identity

Google Cloud

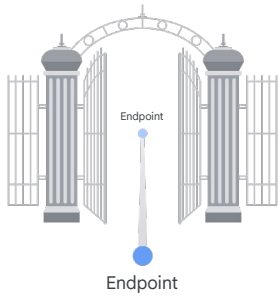
Connect gateway is a GKE feature that simplifies and secures connections and interactions with fleet member clusters, regardless of their location.

Connect gateway builds upon the concept of **Fleets**, which are logical groupings of Kubernetes clusters and other resources that can be managed together.

Let's explore the key features and capabilities of connect gateway.

Proprietary + Confidential

Use connect gateway for centralized management



Connect Gateway

Google Cloud

Permission management can become challenging when working with multiple clusters across different Google Cloud projects.

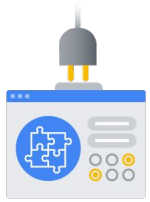
A more centralized and unified approach is to use **connect gateway**, which is discussed in the next lesson.

Proprietary + Confidential

Connect gateway features

Cluster discovery

List and query fleet membership resources to identify registered clusters.



Connectivity

Establish a connection to the cluster's API server.

Authentication

Authenticate using a Google ID or third-party identity providers.

Authorization

Use individual IDs or group memberships for efficient policy management.

Google Cloud

Cluster discovery: You can use the Google Cloud CLI to list and query fleet membership resources to identify registered clusters.

Connectivity: Connect gateway uses connect service to establish a connection to the cluster's API server. This is the same infrastructure used to display registered GKE clusters in the Google Cloud console.

Authentication: Google IDs are the default authentication credentials used by Connect Gateway, but you can also use third-party identity providers through workforce identity federation and group-based authentication through GKE Identity Service.

Authorization: Authorization is consistent across all clusters registered in a fleet, and connect gateway uses individual IDs or group memberships for efficient policy management.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

06 Fleet Workload Identity

Google Cloud

In this section, you learn the essential steps to configure connect gateway, so you can access all your clusters with ease.

Proprietary + Confidential

Configure connect gateway

01 Enable GKE fleet APIs

```
gcloud services enable \
gkehub.googleapis.com
```

02 Register clusters

New clusters

```
gcloud container clusters create \
CLUSTER_NAME \ --enable-fleet
```

Existing clusters

```
gcloud container clusters update \
CLUSTER_NAME \
--enable-fleet
```

03 Grant IAM role

```
gcloud projects \
add-iam-policy-binding PROJECT_ID \
   --member= \
      "user:USER_EMAIL" \
   --role="roles/gkehub.viewer"
```

Google Cloud

The first step is to enable the GKE Fleet APIs for your Google Cloud project.

Next, register your Kubernetes clusters (GKE or external) with a GKE fleet. This establishes the connect gateway connection.

Then configure IAM permissions. Grant the **oles/gkehub.viewer** role to users who will retrieve kubeconfig files, so they can use kubectl through connect gateway.

Proprietary + Confidential

Configure connect gateway

05 Grant RBAC user permissions

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-only-access
  namespace: my-namespace
subjects:
- kind: User
  name: alice@example.com
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: view
  apiGroup: rbac.authorization.k8s.io
```

06 Interact with clusters

```
gcloud container fleet memberships \
get-credentials MEMBERSHIP_NAME \
--project=PROJECT_ID

kubectl get pods \
--namespace=my-namespace
```

Google Cloud

Within each Kubernetes cluster, use RBAC to grant users specific permissions to access Kubernetes resources.

Create RoleBindings for namespace-specific permissions or ClusterRoleBindings for cluster-wide permissions.

After granting the necessary IAM permissions, users can retrieve the kubeconfig file using the `gcloud container fleet memberships get-credentials` command and use kubectl to interact with the registered clusters.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

06 Fleet Workload Identity

Google Cloud

Traditionally, accessing GKE clusters could be a challenge.

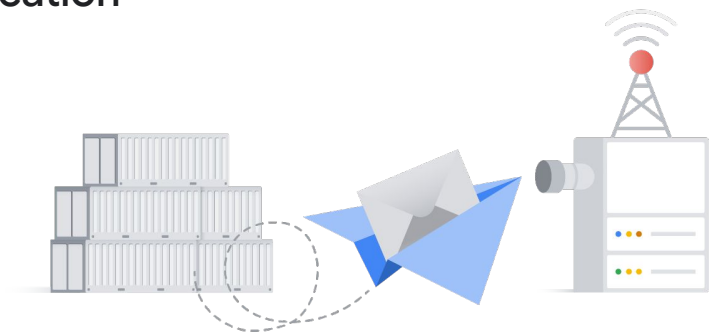
Often kubectl needed to be configured with Google Cloud account authentication using the `gcloud` tool or with Kubernetes Service Account Credentials. This could result in credential management challenges, especially in large organizations with multiple teams and clusters.

GKE Identity Service addresses these challenges by enabling authentication through your existing IdP. This simplifies access management and improves security by centralizing authentication.

So how does GKE Identity Service work?

Proprietary + Confidential

GKE Identity Service uses the OIDC standard for authentication



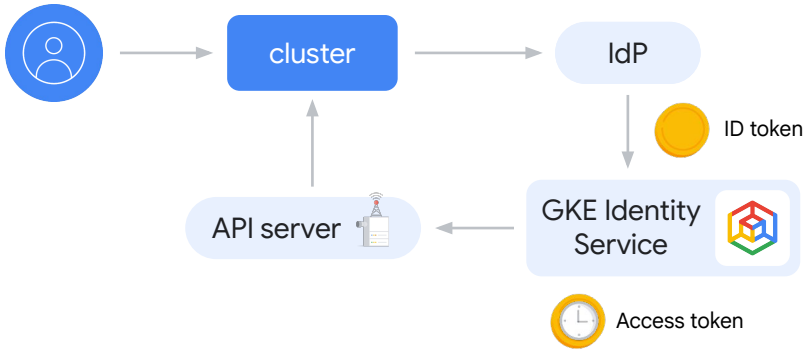
GKE uses OpenID connect (OIDC) to securely and efficiently authenticate to GKE clusters through third-party Identity Providers (IdPs).

Google Cloud

With GKE Identity Service, you can leverage OpenID connect (OIDC) to securely and efficiently authenticate to GKE clusters through third-party Identity Providers (IdPs) such as Okta, Auth0, and Azure Active Directory.

Proprietary + Confidential

How GKE Identity Service works



Google Cloud

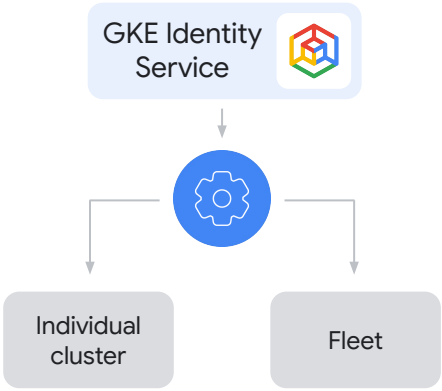
When a user attempts to access a GKE cluster, they are redirected to the IdP for authentication.

After successful authentication, the IdP issues an ID token, which is then exchanged for a temporary access token by GKE Identity Service.

This access token is used to authenticate to the Kubernetes API server.

Proprietary + Confidential

Cluster and fleet-level configuration



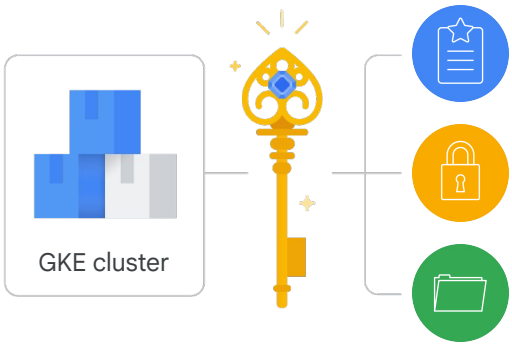
Google Cloud

GKE Identity Service can be configured at the individual cluster level or at the fleet level through connect gateway.

This dual configuration approach offers significant flexibility in managing authentication across diverse GKE environments.

Proprietary + Confidential

GKE Identity Service benefits



- Simplified access management
- Enhanced security
- Centralized identity management
- Improved auditability
- Support for various IdPs

Google Cloud

Finally, let's explore the benefits of GKE Identity Service.

Simplified access management:
With GKE Identity Service, you don't need to manage separate Kubernetes credentials. This reduces administrative overhead and improves the user experience.

Enhanced security: You can leverage your organization's existing IAM infrastructure to provide strong security controls and reduce the risk of unauthorized access.

Centralized identity management: Centrally manage access and permissions to ensure consistency and compliance across your GKE environment.

Improved auditability: Detailed audit logs of user authentication and authorization activities are available for visibility and accountability.

Support for various IdPs: A wide range of OIDC-compliant IdPs gives you the flexibility to choose the solution that best meets your needs.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

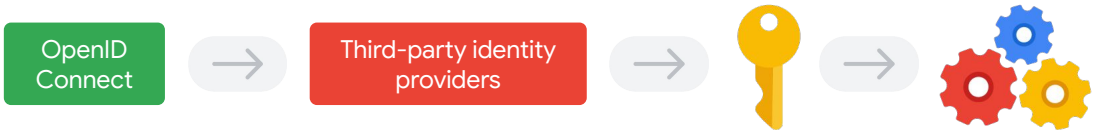
06 Fleet Workload Identity

Google Cloud

With GKE Identity Service, you can use your existing identity provider (**IdP**) to authenticate users and grant access to your GKE clusters, eliminating the need to manage separate user accounts and passwords for each cluster.

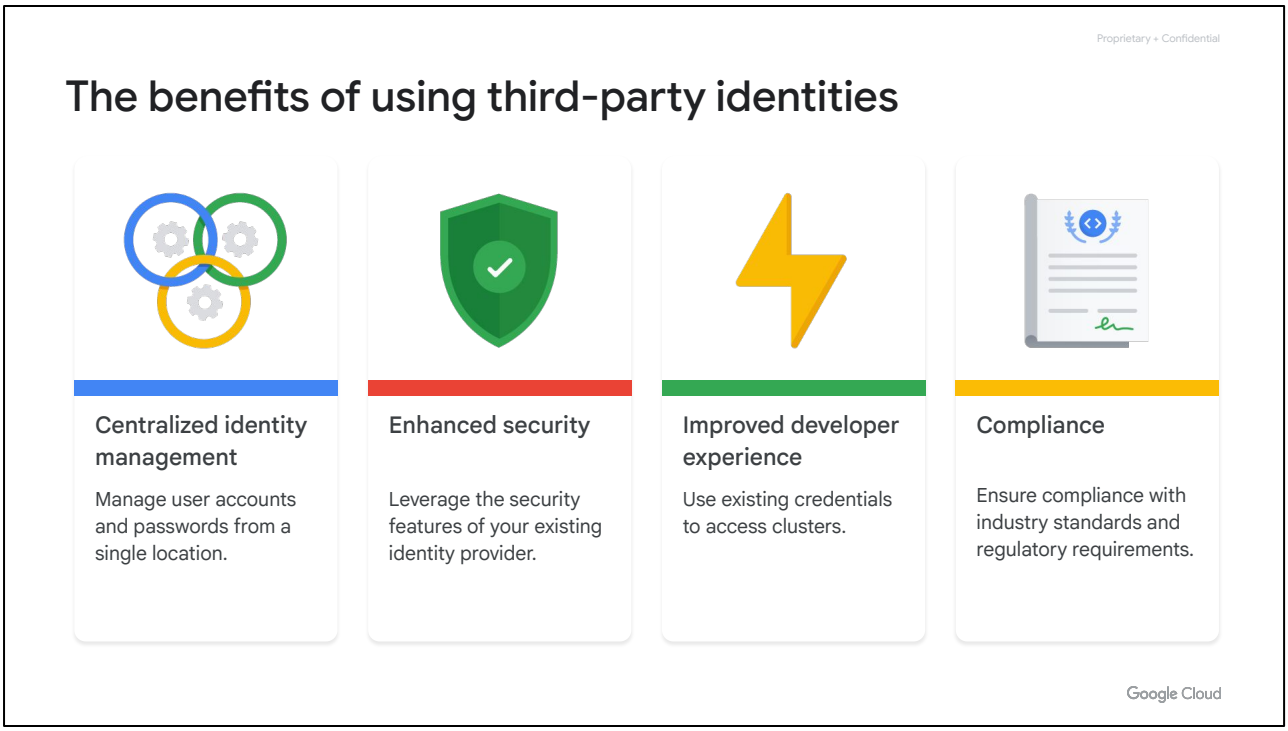
Proprietary + Confidential

GKE Identity Service uses OpenID Connect for authentication



Google Cloud

GKE Identity Service uses OpenID Connect (**OIDC**) for GKE cluster authentication. OpenID Connect enables you to use external identity providers, like Okta, Auth0, and Azure Active Directory to authenticate users and grant access to Kubernetes resources.



There are several benefits to using third-party identities.

Centralized identity management:
Manage user accounts and passwords from a single location, simplifying user management and improving security.

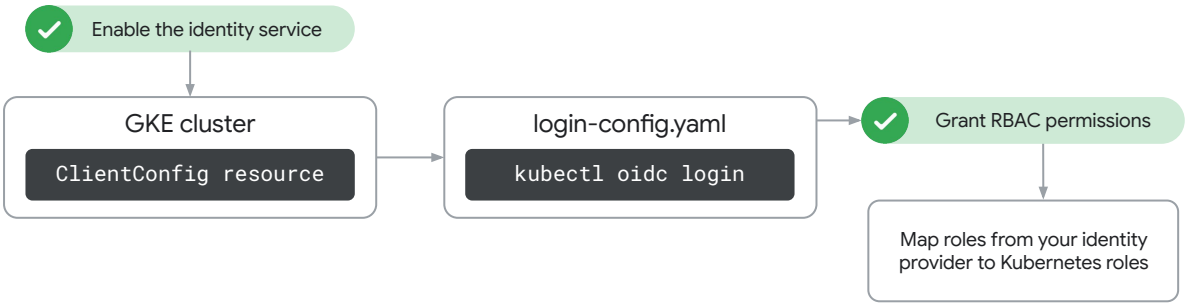
Enhanced security:
Leverage the security features of your existing identity provider, such as multi-factor authentication (MFA) and password policies.

Improved developer experience: Developers can use their existing credentials to access clusters, reducing friction and improving productivity.

Compliance: Use an identity provider that complies with industry standards and meets regulatory requirements.

Proprietary + Confidential

Configure GKE Identity Service to use a third-party identity provider



Google Cloud

Now, how do you configure GKE Identity Service to use an existing third-party identity provider?

First, enable the Identity Service on your GKE cluster.

Then, configure your cluster to trust your third-party identity provider by creating a ClientConfig custom resource in your cluster, containing details about your IdP.

Next, use your IdP to set up authentication for kubectl. This typically involves downloading a login-config.yaml file and using the kubectl oidc login command.

After configuring authentication for kubectl, use Kubernetes RBAC to grant users and groups permissions to cluster resources based on their roles and responsibilities.

Finally, map roles from your identity provider to Kubernetes roles to control access to resources.

Proprietary + Confidential

Best practices for configuring GKE Identity Service

Least privilege

Grant users only the necessary permissions to perform their tasks.

Regular audits

Regularly audit user permissions and revoke access when no longer needed.

Strong passwords

Enforce strong password policies and enable MFA for enhanced security.

Google Cloud

When configuring the GKE Identity Service, it is recommended that you adhere to the following best practices.

Principle of least privilege: Grant users only the necessary permissions to perform their tasks.

Regular audits: Regularly audit user permissions and revoke access when no longer needed.

Strong passwords and MFA: Enforce strong password policies and enable MFA for enhanced security.

Proprietary + Confidential

Troubleshooting

Verify configuration

Ensure your ClientConfig and authentication configuration do not have any errors.

Check logs

Examine the logs of the Identity Service components and your identity provider for troubleshooting information.

Refer to documentation

Refer to the official GKE Identity Service documentation for troubleshooting guides and FAQs.

Google Cloud

Finally, let's cover tips for troubleshooting GKE Identity Service.

Verify configuration by ensuring that your ClientConfig and authentication configuration do not have any errors.

Examine the logs of the Identity Service components and your identity provider for troubleshooting information, and refer to the official GKE Identity Service documentation for troubleshooting guides and FAQs.

Proprietary + Confidential

Manage Identity in GKE with Authentication

01 Introduction to GKE Identity Service

02 Connect gateway overview

03 Configuring connect gateway for authentication and authorization

04 Accessing clusters with GKE Identity Service

05 Authenticating third-party identities with GKE Identity Service

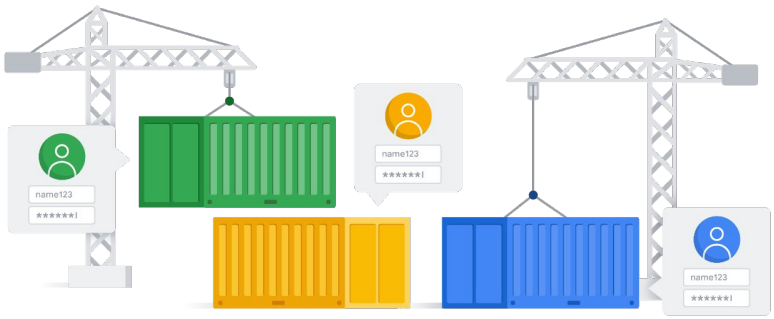
06 Fleet Workload Identity

Google Cloud

Traditionally, Kubernetes applications relied on service accounts with associated secrets to authenticate to Google Cloud services. This method posed security risks as these secrets needed to be managed and securely stored, making them potential targets for attackers. Additionally, managing these secrets across multiple clusters and projects added another layer of complexity.

Proprietary + Confidential

Workload Identity simplifies how applications access Google Cloud services

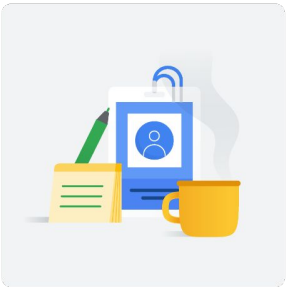


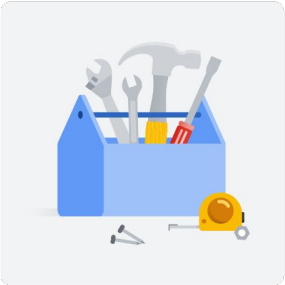
Google Cloud

Workload Identity addresses these challenges by leveraging the Kubernetes service account as the identity for workloads, eliminating the need for managing explicit Google service account keys.

Proprietary + Confidential

Workload Identity establishes a trust relationship between the KSA and GSA



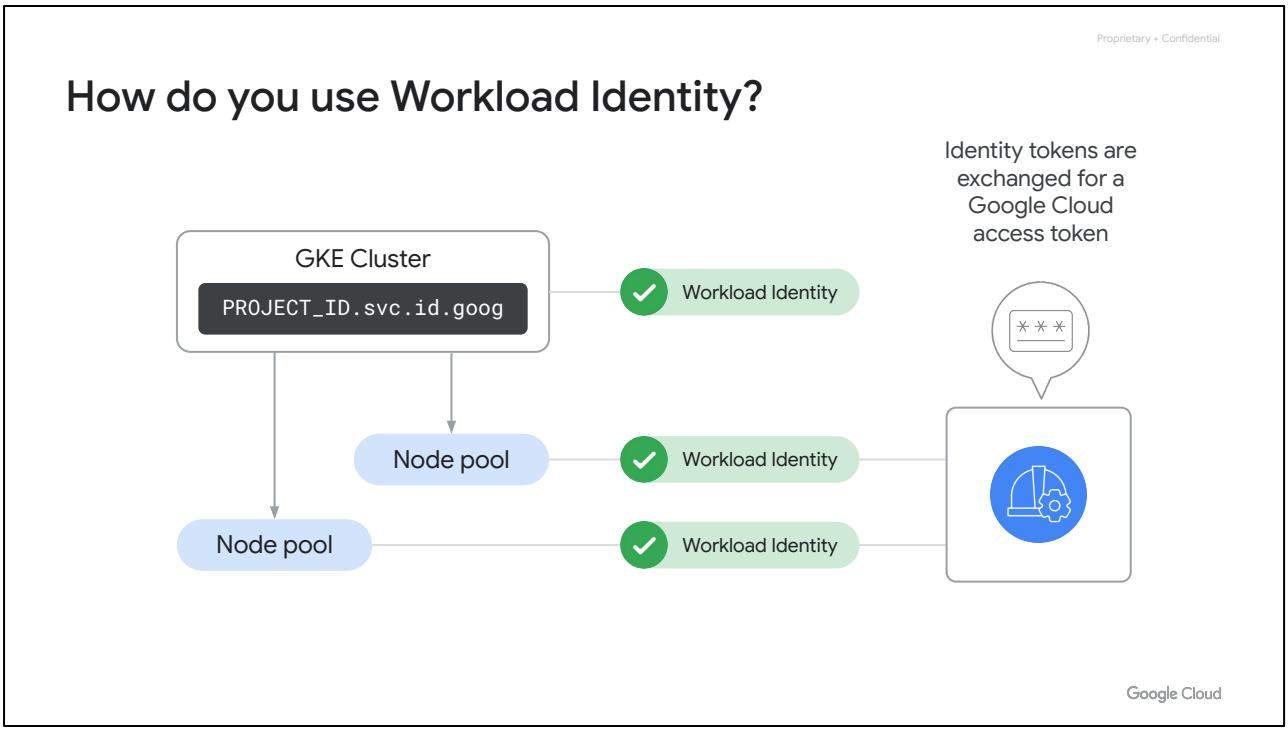


Google Cloud Service Account (GSA)

Kubernetes service accounts (KSA)

Google Cloud

Workload Identity establishes a trust relationship between the Kubernetes service account (KSA) and a Google Cloud service account (GSA), allowing workloads to seamlessly access cloud resources.



How does Workload Identity work?

Each GKE cluster acts as an OpenID Connect (OIDC) identity provider, issuing identities for Kubernetes service accounts.

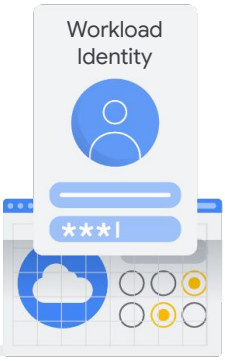
A workload identity pool is created in your Google Cloud project. This pool bridges the gap between Kubernetes and Google Cloud IAM.

Next, a Kubernetes service account is linked to a Google Cloud service account within the workload identity pool.

When a workload needs to access a Google Cloud service, it obtains an identity token from the GKE provider using its Kubernetes service account. This token is then exchanged for a Google Cloud access token, granting the workload the necessary permissions.

Proprietary + Confidential

Workload Identity use cases



- Workload Identity is primarily used for GKE clusters on Google Cloud.
- It is used to grant granular application permissions.
- Workload Identity is commonly used for multi-cluster environments and fleet management.

Google Cloud

Workload Identity is primarily used for GKE clusters on Google Cloud.

For Kubernetes workloads running on GKE, Workload Identity is the recommended method for providing access to other Google Cloud services like Cloud Storage, Cloud SQL, and Pub/Sub.

Workload Identity is secure, easy to manage, and offers high performance.

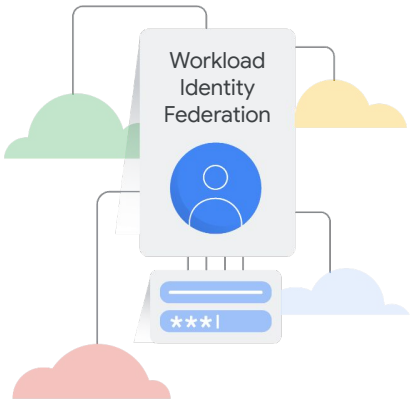
It is also useful when applications require specific permissions on Google Cloud resources. Granular permissions are granted to the GSA, and then the GSA is linked to the KSA.

This process is more secure than granting a single, overly permissive service account key to your entire cluster.

Workload Identity works seamlessly with fleets and provides a consistent way for workloads across multiple clusters within a fleet to access Google Cloud services. This simplifies identity management and enhances security across your entire GKE environment.

Proprietary + Confidential

Workload Identity Federation



- Workload Identity Federation extends to workloads running outside of Google Cloud.
- Workloads running in any environment can access Google Cloud resources without needing to download and manage Google Cloud service account keys.
- Establishes a trust relationship between your external identity provider and Google Cloud.

Google Cloud

Now, let's transition to **Workload Identity Federation**.

Whereas Workload Identity focuses on GKE clusters on Google Cloud, Workload Identity Federation extends to workloads running outside of Google Cloud.

With Workload Identity Federation, workloads running in any environment can access Google Cloud resources without needing to download and manage Google Cloud service account keys.

Workload Identity Federation establishes a trust relationship between your external identity provider (AWS IAM, Azure AD, on-prem IdP, or any other OIDC-compliant provider) and Google Cloud.

Proprietary + Confidential

How do you use Workload Identity Federation?

01 Create a workload identity pool in Google Cloud.

02 Configure a provider and specify IdP details within the identity pool.

03 Map identities from your external IdP to Google Cloud Service Accounts.

04 Workloads exchanges the external IdP token for a temporary Google Cloud access token.

Google Cloud

Let's explore how Workload Identity Federation works.

Your external environment has its own identity provider that issues tokens to workloads.

Create a workload identity pool in Google Cloud.

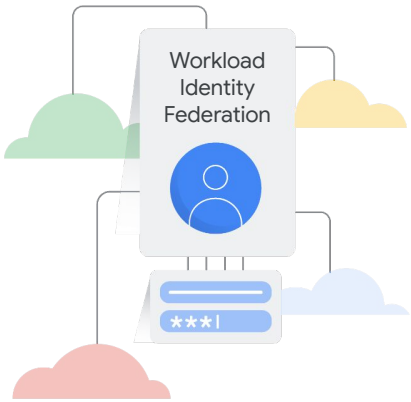
Then, configure a provider within the pool, specifying details about your external IdP (issuer URL, audience, etc.).

Next, define rules to map subjects (identities) from your external IdP to Google Cloud service accounts.

Workloads running externally exchange their external IdP token for a temporary Google Cloud access token.

Proprietary + Confidential

Workload Identity Federation use cases



- Primarily used for hybrid and multi-cloud environments.
- Workloads running outside Google Cloud can interact with Google Cloud services.
- Reduces security risks and simplifies service account key management.
- Commonly used for Kubernetes running on other containerized platforms or on-premises.
- Used for applications that have not fully migrated to Google Cloud.

Google Cloud

Workload Identity Federation is primarily used for hybrid and multi-cloud environments, so that workloads running outside of Google Cloud can interact with Google Cloud services.

Workload Identity Federation reduces security risks and simplifies service account key management.

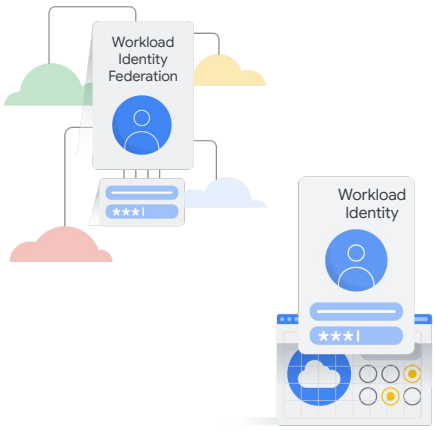
If your application is split between Google Cloud and other providers like AWS or Azure, use Workload Identity Federation to securely and easily grant your workloads on those other clouds access to Google Cloud resources.

Another common use case is Kubernetes running on other containerized platforms or on-premises.

Workload Identity Federation should also be used for applications that have not fully migrated to Google Cloud but must interact with Google Cloud services.

Proprietary + Confidential

Workload Identity and Workload Identity Federation commonalities



- Enhanced security
- Simplified management
- Auditing
- Principle of least privilege
- Namespace-based organization
- Regular audits
- Automation

Google Cloud

Both Workload Identity and Workload Identity Federation have common benefits.

Enhanced security: There is no need to manage long-lived service account keys.

Simplified management: Centralized IAM management in Google Cloud, even for external workloads.

Auditing: Google Cloud logs all access, providing better visibility and control.

Principle of least privilege: Grant workloads only the necessary permissions to access Google Cloud resources.

Namespace-based organization: Use Kubernetes namespaces to logically group workloads and assign appropriate permissions.

Regular audits: Regularly review and audit workload identities and permissions to ensure they are up-to-date and aligned with your security policies.

Automation: Leverage automation tools like Terraform or Config Sync to manage and deploy Workload Identity configurations across your fleet.